EduAirControl — Implementación del software

> Procedimiento de preparación, construcción, configuración, despliegue y verificación del sistema

Servicio Nacional de Aprendizaje · Análisis y Desarrollo de Software – 3145556
Integrantes: María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas
Instructores: Todos los instructores del programa
Versión 1.0 · 9 de octubre de 2026 · 2025-2026

Tabla de contenido

- 1. Introducción
- 2. Objetivo
- 3. Alcance y criterios de lectura
- 4. Requisitos de implementación
  - 4.1 Herramientas
  - 4.2 Requisitos de hardware
  - 4.3 Puertos
- 5. Organización del código
- 6. Configuración de variables de entorno
- 7. Implementación local con Docker Compose
  - 7.1 Preparación
  - 7.2 Construcción y arranque
  - 7.3 Espera y comprobación de servicios
  - 7.4 Stack de observabilidad
- 8. Implementación del frontend web
- 9. Implementación de la aplicación móvil
- 10. Implementación de base de datos y migraciones
- 11. Implementación de la integración IoT

- 12. Integración continua y construcción de imágenes
- 13. Despliegue en Kubernetes
- 14. Verificación y pruebas posteriores
- 15. Despliegue, reversión y recuperación
- 16. Seguridad de la implementación
- 17. Estado de implementación y pendientes
- 18. Entregables y evidencias
- 19. Glosario
- 20. Referencias internas

---

1. Introducción

Este documento describe cómo implementar EduAirControl a partir del repositorio entregado. El sistema integra una aplicación web, una aplicación móvil, un API Gateway, servicios Spring Boot, bases de datos PostgreSQL, Redis, RabbitMQ, herramientas de observabilidad y un prototipo IoT basado en ESP32.

La guía está escrita para un entorno de desarrollo, integración, demostración y preparación de despliegue. La existencia de un Dockerfile, un manifiesto Kubernetes o una configuración de observabilidad significa que el mecanismo está preparado o documentado; no significa automáticamente que se encuentre validado en producción.

2. Objetivo

Definir un procedimiento reproducible para:

- obtener y organizar el código fuente;
- preparar las herramientas y variables de entorno;
- construir y ejecutar los servicios;
- crear y actualizar las bases de datos mediante Liquibase;
- desplegar el frontend web y la aplicación móvil;
- integrar el prototipo ESP32 con DHT11, KY-038 y MQ-135;
- ejecutar las comprobaciones de salud, humo y aceptación;
- publicar imágenes mediante integración continua;
- preparar un despliegue controlado en Kubernetes;

- realizar una reversión o recuperación cuando sea necesario.

3. Alcance y criterios de lectura

Incluido

- Implementación local con `docker-compose.yml` en `microservicios/ea-control-infra`.
- Construcción de `api-gateway`, `ms-security`, `ms-classroom-management`, `ms-sensor-management`, `ms-environment-monitoring`, `ms-user-management` y `ms-user-experience`.
- Bases PostgreSQL independientes por servicio en el stack local.
- Redis para sesión, lista negra y limitación de tasa.
- RabbitMQ para eventos y comunicación asíncrona.
- Frontend web React/Vite servido por Nginx en imagen Docker.
- Aplicación móvil Expo/React Native con aprovisionamiento BLE.
- Manifiestos de despliegue para Kubernetes.
- CI de servicios Java mediante GitHub Actions.
- Pruebas del prototipo con DHT11 y KY-038.

No confirmado como producción

- Firmware definitivo que envía las lecturas del ESP32 a la API.
- Calibración metrológica del DHT11, KY-038 o MQ-135.
- Estrategia productiva definitiva, proveedor cloud, dominio y certificados.
- Cobertura de pruebas medida con JaCoCo.
- Observabilidad completa conectada a métricas efectivas de cada servicio.
- Restauración de copias probada en un entorno operativo.
- Separación final de todos los datos y contratos entre servicios.

> **Regla:** los valores de ejemplo, contraseñas locales, dominios `example.com` y claves de demostración deben reemplazarse antes de cualquier uso institucional o productivo.

4. Requisitos de implementación

4.1 Herramientas

Herramienta	Versión o condición	Uso
Git	2.40 o superior recomendado	Obtener el código y administrar versiones
Docker Desktop	Docker Engine 24 o superior recomendado	Contenedores locales
Docker Compose	2.20 o superior recomendado	Orquestar el stack
Java	JDK 17	Construir y probar servicios Spring Boot
Maven Wrapper	`./mvnw` o `mvnw.cmd`	Compilar cada servicio
Node.js y npm	Node 20 recomendado para el frontend Docker; compatible con Expo según el proyecto	Frontend web y móvil
Android Studio	Para ejecución Android nativa	SDK, emulador y `expo run:android`
Arduino IDE	Para pruebas ESP32	Cargar firmware de prueba
`curl` y `jq`	Disponibles en el equipo	Health checks y smoke test
Kubernetes y kubectl	Solo para despliegue K8s	Aplicar manifiestos y revisar pods

4.2 Requisitos de hardware

- Computador con mínimo recomendado de 8 GB de RAM.
- Espacio para imágenes Docker, dependencias, compilaciones y volúmenes PostgreSQL.

- ESP32.
- DHT11 para temperatura y humedad.
- KY-038 para nivel relativo de sonido.
- MQ-135 para cambios relativos de gases/calidad del aire.
- Protoboard, jumpers y alimentación segura.
- Teléfono Android/iOS o emulador para probar la app móvil.

4.3 Puertos

Servicio	Puerto publicado
API Gateway	8080
`ms-security`	18081 en el host; 8081 dentro de Docker
`ms-classroom-management`	3002
`ms-environment-monitoring`	3003
`ms-sensor-management`	3004
`ms-user-experience`	3006
`ms-user-management`	3007
PostgreSQL security	5437
PostgreSQL classroom	5433
PostgreSQL sensors	5434
PostgreSQL monitoring	5435
PostgreSQL user management	5438
PostgreSQL user experience	5439
Redis	6379
RabbitMQ AMQP	5672
RabbitMQ Management	15672
Prometheus	9090
Grafana	3001
OpenTelemetry OTLP	4317 / 4318

Servicio	Puerto publicado
Frontend web Docker	80 dentro del contenedor; publicar en 3000 o 5173 según el entorno

Antes de iniciar, comprobar que no existan conflictos de puertos. En Windows puede utilizarse `netstat -ano`; en Linux/macOS, `lsof -i :<puerto>`.

5. Organización del código

La entrega contiene dos áreas principales:

```
proyecto/
├── Docs-ea-control/                          # Documentación de ingeniería
├── ea-control-frontend/
│   └── apps/
│       ├── web/                              # React + Vite + Nginx
│       └── mobile/                           # Expo + React Native
├── microservicios/
│   ├── ea-control-api-gateway/
│   ├── ea-control-ms-security/
│   ├── ea-control-ms-classroom-management/
│   ├── ea-control-ms-sensor-management/
│   ├── ms-environment-monitoring/
│   ├── ea-control-ms-user-management/
│   ├── -ms-user-experience/
│   └── ea-control-infra/                     # Compose, K8s, CI y scripts
└── manuales/                                 # Manuales de usuario, técnic
```

La infraestructura central está en `microservicios/ea-control-infra`:

- `docker-compose.yml`: stack principal.
- `docker-compose.observability.yml`: Prometheus, Grafana y OpenTelemetry Collector.
- `scripts/wait-for-services.sh`: espera de endpoints de salud.
- `scripts/smoke.sh`: prueba de registro, login y logout.
- `deploy/k8s/`: namespace, aplicaciones, bases, Redis, RabbitMQ, secretos e ingress.

- `.github/workflows/build-services.yml`: pruebas, construcción y publicación de imágenes.

6. Configuración de variables de entorno

Crear los archivos de entorno a partir de los ejemplos, sin versionar valores reales:

```
cd proyecto/Docs-ea-control
cp .env.example .env

cd ../ea-control-frontend/apps/web
cp .env.example .env

cd ../mobile/EduAirControlMobile/Front-End
cp .env.example .env
```

En el stack de infraestructura, Compose acepta variables como:

Variable	Uso local
`GATEWAY_PORT`	Puerto del API Gateway, por defecto 8080
`CORS_ALLOWED_ORIGINS`	Orígenes del frontend permitidos
`SECURITY_DB`, `SECURITY_USER`, `SECURITY_PASSWORD`	Base y credenciales de seguridad
`CLASSROOM_DB`, `SENSOR_DB`, `MONITORING_DB`	Bases por dominio
`JWT_SECRET`	Validación de servicios que aún la requieren
`RABBITMQ_USER`, `RABBITMQ_PASSWORD`	Credenciales del broker
`REDIS_PORT`	Puerto de Redis
`REGISTRY`, `TAG`	Registro y etiqueta de imágenes
`GRAFANA_USER`, `GRAFANA_PASSWORD`	Acceso local a Grafana

El frontend utiliza `VITE_API_URL=http://localhost:8080`. La aplicación móvil acepta `EXPO_PUBLIC_API_URL`; en un teléfono físico debe apuntar a la IP LAN del computador, por ejemplo `http://192.168.1.20:8080`.

Para generar una clave local temporal puede utilizarse:

```
openssl rand -base64 32
```

No usar las credenciales `guest/guest`, `change-me-in-vault` ni valores predeterminados en producción.

7. Implementación local con Docker Compose

7.1 Preparación

1. Instalar Docker Desktop y comprobar que el motor Linux esté activo.
2. Abrir una terminal en la raíz del proyecto.
3. Revisar que los proyectos de servicios estén ubicados junto a `ea-control-infra`.
4. Crear o revisar el archivo `.env` del stack.
5. Confirmar que las claves, dominios y contraseñas de producción no estén en el repositorio.

7.2 Construcción y arranque

Desde la infraestructura:

```
cd proyecto/microservicios/ea-control-infra
docker compose build
docker compose up -d
```

Para construir y arrancar en un solo paso:

```
docker compose up -d --build
```

El stack construye los servicios Spring Boot y descarga PostgreSQL 15, Redis 7 y RabbitMQ 3.13 Management. Cada servicio de aplicación depende de sus recursos principales y declara un `healthcheck`.

Consultar el estado:

```
docker compose ps
docker compose logs -f api-gateway
docker compose logs -f ms-security
```

7.3 Espera y comprobación de servicios

El script disponible espera por endpoints de salud del gateway, seguridad, aulas y sensores:

```
chmod +x scripts/wait-for-services.sh scripts/smoke.sh
./scripts/wait-for-services.sh
```

El tiempo máximo se puede modificar:

```
TIMEOUT=300 ./scripts/wait-for-services.sh
```

El smoke test requiere `jq` y debe ejecutarse solo en una base de datos de desarrollo, porque crea un usuario de prueba:

```
./scripts/smoke.sh
```

Comprobación manual mínima:

```
curl -fsS http://localhost:8080/health
curl -fsS http://localhost:8081/api/v1/auth/health
curl -fsS http://localhost:3002/health
curl -fsS http://localhost:3004/health
```

7.4 Stack de observabilidad

Para iniciar el stack principal y la observabilidad:

```
docker compose \
  -f docker-compose.yml \
  -f docker-compose.observability.yml \
  up -d --build
```

Accesos locales:

- Prometheus: `http://localhost:9090`.
- Grafana: `http://localhost:3001`.
- OTLP HTTP: `http://localhost:4318`.

La configuración está preparada en `observability/prometheus.yml` y `observability/otel-collector.yaml`. La disponibilidad de métricas efectivas por servicio debe comprobarse antes de declarar observabilidad completa.

Para detener sin eliminar volúmenes:

```
docker compose down
```

Para eliminar contenedores y datos locales —acción destructiva—:

```
docker compose down -v
```

8. Implementación del frontend web

Ejecución en desarrollo

```
cd proyecto/ea-control-frontend/apps/web
npm install
npm run dev
```

Vite normalmente publica el frontend en `http://localhost:5173`. La variable `VITE_API_URL` debe apuntar al gateway.

Construcción de producción

```
npm run lint
npm run build
npm run preview
```

La imagen Docker usa dos etapas: Node 20 construye la aplicación y Nginx sirve `dist`. Para crearla manualmente:

```
docker build \
  --build-arg VITE_API_URL=http://localhost:8080 \
  -t eduaircontrol-frontend:local .
```

La configuración Nginx usa `try_files` para que las rutas de React funcionen al recargar la página. El puerto interno de la imagen es 80; el puerto externo se define en el despliegue que la consuma.

9. Implementación de la aplicación móvil

La aplicación se encuentra en `ea-control-frontend/apps/mobile/EduAirControlMobile/Front-End`.

```
cd proyecto/ea-control-frontend/apps/mobile/EduAirControlMobile/Front-End
npm install
npm start
```

Comandos disponibles:

```
npm run android
npm run ios
npm run prebuild
npm run check:android
npm run lint
```

La aplicación usa Expo 54.0.37, React Native 0.81.5 y React 19.1.0. Para un teléfono físico:

```
EXPO_PUBLIC_API_URL=http://192.168.1.X:8080 npx expo start
```

En el emulador Android, `10.0.2.2` representa al computador anfitrión. El aprovisionamiento BLE busca el dispositivo `ESP32_Config`; Android 12 o superior requiere permisos `BLUETOOTH_SCAN` y `BLUETOOTH_CONNECT`.

10. Implementación de base de datos y migraciones

El Compose principal levanta una base PostgreSQL por dominio:

- `eduaircontrol_security`.
- `eduaircontrol_classrooms`.
- `eduaircontrol_sensors`.
- `eduaircontrol_monitoring`.
- `eduaircontrol_user_management`.
- `eduaircontrol_user_experience`.

Cada servicio ejecuta sus changelogs Liquibase al iniciar. Los cambios se encuentran en `src/main/resources/db/changelog` de cada servicio. Hibernate utiliza validación del

esquema; no se debe crear o modificar tablas manualmente para sustituir las migraciones.

Procedimiento recomendado:

1. Iniciar primero las bases y esperar su estado `healthy`.
2. Iniciar los servicios de aplicación.
3. Revisar los logs de Liquibase.
4. Confirmar que no haya errores de checksum o cambios sin aplicar.
5. Ejecutar una prueba de lectura y escritura de cada dominio.
6. Respaldar antes de cambios estructurales.

Los volúmenes persistentes son `security_data`, `classroom_data`, `sensor_data`, `monitoring_data`, `user_management_data`, `user_experience_data` y `rabbitmq_data`. No ejecutar `docker compose down -v` si se deben conservar datos.

11. Implementación de la integración IoT

11.1 Componentes

Componente	Función
ESP32	Controlador, lectura y comunicación Wi-Fi/BLE
DHT11	Temperatura y humedad del aire
KY-038	Nivel relativo de sonido mediante su salida analógica
MQ-135	Cambios relativos de gases/calidad del aire
Protoboard y jumpers	Montaje del prototipo

En los programas de prueba entregados se documentan:

- DHT11 DATA en GPIO 5/D5.
- KY-038 AO en GPIO 34/D34.
- Monitor serial a 115200 baudios.

11.2 Secuencia de implementación

1. Montar el ESP32 y los sensores en la protoboard.
2. Verificar tierra común, alimentación y continuidad.
3. Cargar y ejecutar primero la prueba del DHT11.
4. Cargar y ejecutar la prueba del KY-038.
5. Evaluar el MQ-135 únicamente con adaptación de voltaje adecuada.
6. Definir el firmware único y el contrato real de ingesta.
7. Registrar el dispositivo en el servicio de sensores.
8. Aprovisionar Wi-Fi mediante BLE desde la aplicación móvil si el firmware lo soporta.
9. Enviar una medición de prueba con credencial individual del dispositivo.
10. Comprobar que la medición sea validada, almacenada y visible en el dashboard.

> El DHT11 y el KY-038 cuentan con programas de prueba. El firmware final que conecta el dispositivo con la API, la conexión definitiva del MQ-135, el voltaje de la batería y la calibración siguen pendientes de confirmación. El KY-038 no debe presentarse como sonómetro calibrado y el MQ-135 no debe presentarse como medidor preciso de CO_2 en ppm.

12. Integración continua y construcción de imágenes

El workflow `microservicios/ea-control-infra/.github/workflows/build-services.yml` se ejecuta en `push` a `main`, pull requests hacia `main` y ejecución manual.

La matriz actual incluye seis servicios:

- `api-gateway`.
- `ms-security`.
- `ms-classroom-management`.
- `ms-sensor-management`.
- `ms-user-management`.
- `ms-environment-monitoring`.

Cada servicio configura JDK 17, caché Maven y ejecuta `./mvnw -B verify`. En un `push` a `main`, el workflow construye y publica imágenes en GHCR con las etiquetas `latest` y el SHA del commit.

> `ms-user-experience` existe en el Compose y en el código, pero no aparece en la matriz actual del workflow. Antes de declarar CI completa, se debe incluir y verificar ese servicio.

13. Despliegue en Kubernetes

Los manifiestos se encuentran en `microservicios/ea-control-infra/deploy/k8s`.

Preparación

1. Disponer de un clúster Kubernetes y un Ingress Controller Nginx.
2. Crear el namespace de EduAirControl.
3. Configurar secretos reales fuera del repositorio.
4. Publicar imágenes en un registro accesible por el clúster.
5. Reemplazar las etiquetas de ejemplo y el dominio del Ingress.
6. Configurar TLS mediante cert-manager o el mecanismo institucional.

Aplicación de manifiestos

```
cd proyecto/microservicios/ea-control-infra
kubectl apply -f deploy/k8s/namespace.yaml
kubectl apply -f deploy/k8s/secrets.example.yaml
kubectl apply -f deploy/k8s/configmap.yaml
kubectl apply -f deploy/k8s/postgres.yaml
kubectl apply -f deploy/k8s/redis.yaml
kubectl apply -f deploy/k8s/rabbitmq.yaml
kubectl apply -f deploy/k8s/apps.yaml
kubectl apply -f deploy/k8s/ingress.yaml
```

Verificar el despliegue:

```
kubectl get pods -n eduaircontrol
kubectl get svc -n eduaircontrol
kubectl get ingress -n eduaircontrol
kubectl rollout status deployment/api-gateway -n eduaircontrol
kubectl logs deployment/api-gateway -n eduaircontrol
```

Los manifiestos incluyen réplicas, `readinessProbe`, `livenessProbe`, solicitudes y límites de recursos. No se debe utilizar `secrets.example.yaml` con contraseñas reales. La configuración K8s debe revisarse para confirmar que todos los servicios presentes en Compose estén también desplegados, especialmente `ms-user-experience`.

14. Verificación y pruebas posteriores

La implementación se considera técnicamente aceptable cuando se cumplen como mínimo:

Verificación	Criterio
Contenedores	Todos los servicios requeridos están `running` y sus health checks responden
Gateway	`GET /health` responde satisfactoriamente
Seguridad	Registro, login, renovación y logout funcionan en ambiente de prueba
Gateway	Las rutas se enrutan al servicio correcto
Base de datos	Liquibase termina sin errores y los datos persisten después de reiniciar
Eventos	RabbitMQ recibe y entrega eventos sin efectos duplicados
Frontend	`npm run lint` y `npm run build` terminan correctamente
Móvil	Expo inicia y puede comunicarse con el gateway
IoT	DHT11 y KY-038 entregan valores de prueba; el firmware de ingesta se valida por separado
Smoke test	`scripts/smoke.sh` termina sin error en una base de desarrollo
CI	Todos los servicios incluidos en la matriz ejecutan `mvnw verify`
Kubernetes	Pods listos, probes correctas e Ingress accesible en entorno controlado

Las métricas de cobertura, disponibilidad, rendimiento y recuperación no deben declararse cumplidas sin la evidencia correspondiente.

15. Despliegue, reversión y recuperación

Detención segura

```
docker compose stop
```

Reinicio de un servicio

```
docker compose restart ms-environment-monitoring
```

Reconstrucción después de un cambio

```
docker compose up -d --build ms-environment-monitoring
```

Reversión en Kubernetes

```
kubectl rollout undo deployment/api-gateway -n eduaircontrol
kubectl rollout status deployment/api-gateway -n eduaircontrol
```

Antes de una migración de base de datos:

1. detener o aislar el tráfico de escritura;
2. crear una copia de seguridad de las bases afectadas;
3. ejecutar la migración en un entorno de prueba;
4. verificar conteos, índices y restricciones;
5. desplegar el código compatible;
6. mantener un plan de reversión.

La reversión de una migración incompatible no debe ejecutarse automáticamente; debe ser aprobada por el responsable técnico después de evaluar pérdida o transformación de datos.

16. Seguridad de la implementación

- No publicar archivos `.env`, claves privadas, tokens, contraseñas ni secretos Kubernetes.
- Sustituir las claves RSA de demostración y rotarlas ante cualquier exposición.
- Utilizar `CORS_ALLOWED_ORIGINS` con orígenes explícitos.

- Mantener los puertos internos en la red privada cuando sea posible.
- Aplicar HTTPS en accesos externos.
- Usar una credencial individual por dispositivo IoT, con rotación y revocación.
- Validar la firma JWT en cada servicio propietario, no solo en el gateway.
- Restringir los endpoints de métricas a la red interna.
- Revisar imágenes Docker y dependencias antes de liberar.
- No usar los valores `guest`, `dev` o `change-me-in-vault` en producción.

17. Estado de implementación y pendientes

Área	Estado identificado	Acción pendiente
Código de microservicios	Disponible en el ZIP	Confirmar versiones y repositorios canónicos
Docker Compose	Preparado para stack local completo	Ejecutar prueba limpia y documentar resultados
PostgreSQL/Liquibase	Changelogs por servicio disponibles	Probar restauración y migraciones hacia atrás
Redis/RabbitMQ	Configurados en Compose	Validar resiliencia y duplicados
Frontend web	Código, Dockerfile y Nginx disponibles	Confirmar puerto externo final
Aplicación móvil	Expo, BLE y servicios disponibles	Validar en dispositivo físico Android/iOS
CI	Workflow para seis servicios	Incluir `ms-user-experience` y revisar contratos
Kubernetes	Manifiestos disponibles	Revisar imágenes, secretos, dominio, TLS y servicios faltantes
Observabilidad	Compose y archivos de configuración disponibles	Confirmar métricas reales, paneles y alertas
DHT11/KY-038	Programas de prueba disponibles	Integrar firmware definitivo y validar cadena sensor → API
MQ-135	Componente contemplado	Confirmar divisor, alimentación, calibración y uso final
Seguridad	Controles documentados; hallazgos abiertos en calidad	Cerrar AC-01, AC-02 y AC-03 antes de producción

18. Entregables y evidencias

Para cerrar una implementación se recomienda conservar:

- salida de `docker compose ps`;
- logs de arranque sin errores de Liquibase;
- respuesta de los endpoints de salud;
- reporte del smoke test;
- resultado de `npm run lint` y `npm run build`;
- resultado de `./mvnw verify` por servicio;
- evidencia del login y de la consulta de ambientes;
- evidencia de una lectura DHT11 y una lectura KY-038;
- evidencia de la API recibiendo la medición IoT, cuando el firmware esté listo;
- manifiestos y salida de `kubectl get pods` en K8s;
- copia de seguridad y registro de restauración de prueba;
- versión, commit, fecha, responsable y ambiente de cada entrega.

19. Glosario

Término	Definición
API Gateway	Punto de entrada que enruta solicitudes hacia los servicios.
CI/CD	Integración y entrega/despliegue continuos.
Docker Compose	Herramienta para ejecutar varios contenedores coordinadamente.
Health check	Comprobación que indica si un servicio está disponible.
Ingress	Entrada HTTP/HTTPS de Kubernetes hacia los servicios.
Liquibase	Herramienta de versionado y ejecución de cambios de base de datos.
RabbitMQ	Broker de mensajería para eventos asíncronos.
Redis	Almacén en memoria usado para estado y capacidades transversales.
Smoke test	Prueba breve de los flujos esenciales después de implementar.
BLE	Bluetooth Low Energy utilizado para configurar el ESP32.
DHT11	Sensor económico de temperatura y humedad relativa.
KY-038	Módulo de detección de sonido con salida relativa; no es un sonómetro calibrado.
MQ-135	Sensor de gases que requiere calibración y adaptación de voltaje.
RTO/RPO	Objetivos de tiempo de recuperación y pérdida máxima aceptable de datos.

20. Referencias internas

- `Docs-ea-control/10-devops/local-setup.md` — preparación local.
- `Docs-ea-control/10-devops/environments.md` — ambientes y pipeline documentados.
- `Docs-ea-control/05-architecture/overview.md` — arquitectura y tecnologías.

- `Docs-ea-control/05-architecture/decisions/records/ADR-010-containerization-strategy.md` — estrategia de contenedores.
- `Docs-ea-control/05-architecture/decisions/records/ADR-011-production-deployment-strategy.md` — estrategia de despliegue productivo.
- `microservicios/ea-control-infra/docker-compose.yml` — stack local.
- `microservicios/ea-control-infra/docker-compose.observability.yml` — observabilidad local.
- `microservicios/ea-control-infra/scripts/wait-for-services.sh` — espera de servicios.
- `microservicios/ea-control-infra/scripts/smoke.sh` — smoke test.
- `microservicios/ea-control-infra/.github/workflows/build-services.yml` — integración continua.
- `microservicios/ea-control-infra/deploy/k8s/` — manifiestos Kubernetes.
- `08_Calidad_Mejoras` — acciones correctivas, preventivas y de mejoramiento.
- `04_Propuesta_Tecnica` — arquitectura, riesgos y decisiones técnicas.